

# **Per-Schema Fee Markets for Attestation-Native Chains**

**EIP-1559-Style Per-Schema Base Fees with PoUA-Coupled Burn**

**Ligate Labs Research, Working Paper v0.2**

**Date:** 2026-05-22

**Contact:** [hello@ligate.io](mailto:hello@ligate.io)

## Abstract

A unified fee market across all transactions assumes homogeneous demand. Attestation-native chains break that assumption: a high-throughput schema (AI-provenance receipts at millions of attestations per day) and a low-throughput, high-value schema (sovereign-identity proofs at hundreds per day) have fundamentally different demand profiles, fee elasticities, and inclusion preferences. A single base-fee curve cannot fit both regimes without penalizing one to serve the other.

This paper proposes **per-schema fee markets** as a runtime primitive on Ligate Chain: each registered schema  $\sigma$  carries its own fee-market state  $(b_\sigma, u_\sigma, T_\sigma)$ , with EIP-1559-style base-fee adjustment, a per-schema tip floor, and a configurable burn-versus-routing split. The mechanism is composable with **sponsored-gas patterns** (Iris MCP relay paying fees on behalf of autonomous agents) and **native delegation** (companion paper): signer and fee-payer are independent fields on the same transaction, and authorization checks for each compose orthogonally.

Three contributions. First, we specify the protocol-level mechanism (§4): the per-schema base-fee adjustment  $b_\sigma(t+1) = b_\sigma(t) \cdot (1 + \xi \cdot (u_\sigma - T_\sigma) / T_\sigma)$ , the tip auction within schema capacity, the burn split  $\text{burn}(\sigma) = \tau_{\text{burn}} + (1 - \tau_{\text{burn}})(1 - \rho_\sigma)$ , and the integration with PoUA’s adaptive  $\tau_{\text{burn}}$  rebase. Second, we prove that the per-schema isolation preserves **PoUA Lemma 1’s cost-to-grind floor** even at the most aggressive routing ( $\rho_\sigma = 0.5$  for every schema): the burned fraction per attestation remains  $\geq \tau_{\text{burn}} \cdot 0.5 \cdot b_\sigma$ , and the chain-level cost-to-grind formula  $F_{\text{net}} \geq \tau_{\text{burn}} \cdot \Delta r / (\eta \cdot \alpha_{\text{eff}})$  holds per-schema with the same constants. Third, we recommend a v0 calibration  $(\xi, T_\sigma, \rho_\sigma) = (1/8, 0.5, 0)$  with profile-specific deviations for bursty, latency-critical, and high-value-low-volume schemas (§4.2 calibration table).

The mechanism’s central design choice is to make the **schema** the fee-market unit. The chain already accounts for attestation work per-schema (PoUA §4.3 reputation update is schema-tagged). Adding per-schema fee state is a natural extension: state already exists, demand profiles differ by orders of magnitude per-schema, and per-schema isolation means a high-utilization schema cannot drag down the base fee of an unrelated schema. The §6 comparison positions per-schema fees against EIP-1559, Solana priority fees, Cosmos fee market module, and Aptos gas pricing; the §7 implementation walkthrough connects to ligate-chain’s Sovereign SDK integration; the §5 security analysis bounds four cross-schema attack patterns.

---

## 1. Introduction

### 1.1 The Heterogeneous-Demand Thesis

Blockchain fee market design has, until recently, assumed a single workload. Ethereum, the canonical example, fits this assumption: token transfers, DeFi interactions, and contract calls share enough demand-curve shape that a single base fee with global adjustment dynamics serves them adequately. EIP-1559’s unified base fee adjusts to chain-wide congestion. The cost of mixing workloads is paid by everyone uniformly, which is acceptable when no workload is structurally different from the others.

Attestation-native chains break that assumption decisively. Consider two schemas that might co-exist on Ligate Chain at maturity. **Themisra proof-of-prompt/v1** is the AI-provenance receipt for every Claude / ChatGPT / Gemini interaction: at maturity, millions of attestations per day, smoothly distributed, each attestation worth fractions of a cent in fee. **Sovereign-identity proofs**

are government-issued identity assertions: hundreds per day, sparse, each attestation potentially gating access to financial services, government benefits, or compliance reporting and therefore worth dollars in fee. The two schemas share a chain but share nothing about their demand curves: volume differs by four orders of magnitude, fee elasticity differs by three, time-of-day distribution differs in shape entirely.

A single base fee cannot serve both. If the chain prices for the high-volume schema, the identity schema either underpays (and its bandwidth is squeezed by transient surges in the AI-provenance workload) or overpays (and the chain extracts more from identity users than the workload justifies). If the chain prices for the identity schema, the high-volume schema’s users see fee swings driven by other workloads they have nothing to do with. The misalignment cost compounds at scale.

The thesis of this paper: the **schema is the natural fee-market unit on attestation-native chains**. The chain already accounts per-schema for attestation work (PoUA §4.3 reputation update is schema-tagged, schema-bound tokens depend on per-schema attester sets, the schema registration mechanism creates first-class chain state per schema). Per-schema fee state is a natural extension of state that already exists, and per-schema isolation means a congested schema’s base fee no longer drags an uncongested schema’s users.

## 1.2 Why Now

Three independent developments converge to make per-schema fee markets implementable and desirable in 2026.

**EIP-1559 is mature.** Ethereum mainnet has shipped EIP-1559 since August 2021. The base-fee adjustment formula  $b(t+1) = b(t) \cdot (1 + \xi \cdot (u - T)/T)$  with  $\xi = 1/8$  has been live in production for five years. Convergence behavior, attack-surface, and parameter calibration are well-understood. We can borrow the dynamics and apply them per-schema with high confidence in stability.

**Multi-resource fee markets are an active research area.** Solana’s priority fees, Cosmos’s `x/feemarket` module, Aptos’s gas pricing, and the EIP-7732 / EIP-4844 multi-dimensional fee proposals on Ethereum have each explored varieties of multi-resource pricing. The conceptual space is mapped; what is missing is a chain that treats workload type (rather than compute / storage / DA) as the resource being priced. That is what per-schema fees does.

**Attestation chains have per-schema state at the consensus layer.** PoUA’s reputation accounting is schema-tagged. Schema-bound tokens use per-schema attester sets as mint authority. Native delegation grants are schema-scoped (per the companion paper §3.3). Adding fee-market state per-schema is the natural completion of a state model that is already per-schema everywhere else.

**Iris is the bellwether product.** Iris’s USD-billed-relayer model needs predictable per-schema fee dynamics: an Iris customer running a Themisra session needs to know what the per-attestation cost will be over the next 24 hours, not what the chain-wide average will be. A unified fee market makes Iris’s subscription pricing harder; a per-schema fee market lets Iris hedge per-schema. The product-economic case for per-schema fees is concrete and immediate.

## 1.3 The Misalignment Problem

A unified base fee, like EIP-1559’s, adjusts based on chain-wide utilization. The mechanism’s elegance comes from a single signal: when the chain is congested, prices climb; when it is empty,

prices drop. The economic intuition is clear.

The same mechanism, applied to a chain with heterogeneous workloads, produces three failure modes.

**Failure mode 1: high-volume schema absorbs spikes in low-volume schema.** Suppose the chain runs a steady Themisra workload at 80% utilization (well above the 50% target). The base fee climbs to extract revenue and discourage congestion. Now an identity-proof spike happens (a wave of new-user onboarding flows after a regulatory event). The spike adds 5% to chain-wide utilization, pushing it to 85%. Under a unified fee market, the base fee climbs further. Themisra users, who had nothing to do with the identity spike, pay more because of unrelated activity.

**Failure mode 2: low-volume schema cannot trigger its own price discovery.** Same chain, reverse problem. The identity workload spikes briefly (a few hundred attestations in a short window), but the chain-wide utilization barely moves because Themisra dominates the denominator. The identity spike never gets the price signal it needs to clear quickly. Identity users wait. The high-value low-volume schema does not get bandwidth proportional to its willingness to pay.

**Failure mode 3: aggregate base fee fits no profile.** The chain’s steady-state base fee, optimized for an average across schemas, is too high for the high-volume low-margin schema and too low for the low-volume high-value schema. Both workloads pay the wrong price all the time, with the wrong price defined as the price the workload would have paid under a fee market sized to its own demand profile.

All three failure modes share a structural cause: the chain has more information about workload heterogeneity than the unified fee market can express. Per-schema state already exists in the chain; the fee market should use it.

## 1.4 The Central Question

**How can an attestation-native chain price attestation work in a way that respects per-schema demand heterogeneity, while preserving PoUA’s cost-to-grind security floor and composing cleanly with sponsored-gas and native-delegation patterns?**

This paper answers: per-schema EIP-1559 with a configurable burn-versus-routing split, coupled to PoUA’s adaptive  $\tau_{\text{burn}}$  rebase so that the chain-wide cost-to-grind floor is preserved even under per-schema isolation.

## 1.5 Approach in Brief

Each registered schema carries its own fee-market state: base fee  $b_\sigma$ , current utilization  $u_\sigma$ , target utilization  $T_\sigma$ , plus a small parameter vector for adjustment rate, tip floor, base-fee clip bounds, and a routing fraction  $\rho_\sigma$ . Per-block, the chain adjusts each schema’s base fee independently using the EIP-1559 formula, scoped to that schema’s observed utilization. Tips compete within schema capacity; cross-schema isolation is preserved by allocating per-block attestation slots per schema (proposer chooses the per-block allocation under governance-tuned bounds).

The burn split is the bridge to PoUA. Of every paid base fee, fraction  $\tau_{\text{burn}}$  is burned (the same chain-wide burn that PoUA §4.4 governs), fraction  $(1 - \tau_{\text{burn}}) \cdot \rho_\sigma$  flows to the schema registrant, and fraction  $(1 - \tau_{\text{burn}}) \cdot (1 - \rho_\sigma)$  flows to the validator who included the attestation. Tips flow entirely

to the validator. Section 5 shows that even at the most aggressive routing ( $\rho_\sigma = 0.5$  chain-wide), PoUA’s Lemma 1 cost-to-grind floor is preserved with the same constants.

The mechanism composes orthogonally with two other primitives. **Sponsored gas** lets a third party pay base fee and tip on behalf of an attester (Iris’s pattern): signer and fee-payer are independent transaction fields, and the §4.3 sponsored-gas accounting handles the variance hedging. **Native delegation** (companion paper) lets a master key authorize a hot key to sign attestations within bounded scope; the per-schema fee market does not change anything about who can sign, only about who pays and how the chain prices the work.

## 1.6 Contributions

The paper makes four contributions.

A **mechanism specification** in §4: the per-schema base-fee adjustment formula, the per-block target-utilization calibration table, the tip auction within schema capacity, the burn-and-routing split, and the integration points with PoUA reputation accounting. The mechanism is fully specified at a level where a chain implementer can build it without further ambiguity.

A **cost-to-grind preservation theorem** in §5: under the per-schema fee market with arbitrary schema-set  $\Sigma$  and routing fractions  $\{\rho_\sigma\}_{\sigma \in \Sigma}$  bounded to  $\rho_\sigma \leq 0.5$ , PoUA’s §5.5.3 Lemma 1 holds with the same constants. The chain’s economic-security floor is unchanged by introducing per-schema fees, even at the most adversary-friendly routing setting.

A **comparative analysis** in §6 positioning per-schema fees against EIP-1559, Solana priority fees, Cosmos `x/feemarket`, and Aptos gas pricing across five axes (granularity of pricing, target-utilization mechanism, burn fraction, sponsor-friendly extensions, multi-workload accommodation). Per-schema fees occupies a distinct point in this design space: the only mechanism in the comparison that treats workload type as the unit of pricing.

A **calibration recommendation** in §4.2 (taxonomy of three demand profiles plus latency-critical addendum) and §7.2 (v0 protocol parameters). The recommendation is governance guidance, not a hard constraint; the chain enforces a  $[0.1, 0.9]$  bound on  $T_\sigma$  and a  $[0, 0.5]$  bound on  $\rho_\sigma$ , but schemas declare their own values at registration within those bounds.

**1.6.1 Status of Claims** Following PoUA v0.7’s discipline of separating claim categories explicitly:

**Proven** (formal mathematical argument under standard cryptographic and BFT assumptions):

- The §5.1 cost-to-grind preservation theorem proves PoUA’s Lemma 1 floor holds under per-schema fees with  $\rho_\sigma \leq 0.5$ , with the same constants and the same security argument.

**Empirical or heuristic**, requiring devnet validation:

- The §4.2 calibration table (recommended  $T_\sigma$  per demand profile) is an architectural default. Real-world demand curves for live schemas will refine these targets once devnet operation produces data.
- The §5.4 sponsored-gas adversarial model assumes a specific Iris pricing pattern (pre-committed subscription with retroactive reimbursement above threshold). Other sponsorship models exist and need their own analysis.

**Bounded under stated assumptions**, where the assumptions are non-trivial and named:

- The §5.2 cross-schema censorship bound assumes the PoUA §A.1 KL-divergence detector’s tolerance is calibrated against the chain-wide null distribution of schema arrivals. If the calibration is too loose, cross-schema censorship goes undetected; if too tight, honest fee-driven preference is flagged. Default tolerance from PoUA v0.7 §A.4 holds; per-schema fees do not require recalibration.
- The §3.2 validator income decomposition assumes the proposer is the validator who includes the attestation; in a separated-proposer-and-builder architecture (e.g., MEV-Boost-style), the income flows differently. The mechanism still works but §6.2 needs a separate treatment.

## 1.7 Scope and Non-Goals

### In scope:

- Per-schema fee dynamics: base-fee adjustment, target utilization, tip auction, burn split, fee routing
- Integration with PoUA reputation accounting and adaptive  $\tau_{\text{burn}}$  rebase
- Cost-to-grind preservation under per-schema isolation
- Composition with sponsored gas (Iris paymaster pattern)
- Composition with native delegation (companion paper)
- Cross-schema attack analysis and detector calibration

### Explicitly out of scope:

- **Multi-resource fee markets.** A separate paper would price gas / storage / DA bandwidth as distinct resources. This paper prices workload-type (the schema) as a single composite resource and treats other resources as derived.
- **MEV (maximal extractable value).** Block-builder ordering preferences, sandwich attacks, and reorg incentives are outside this paper. The §6.2 builder-side incentive analysis touches MEV briefly but defers detailed treatment.
- **Cross-chain fee-market portability.** A schema registered on Ligate Chain that is also recognized on a counterparty chain via IBC needs cross-chain price discovery. Out of scope for v0.2; a follow-on paper will cover it.
- **EVM-execution fee market.** If Ligate Chain’s v4 phase ships EVM execution (per the chain roadmap, deliberately deferred), EVM contract calls will have their own fee dynamics that this paper does not address.

## 1.8 Document Structure

Section 1.6.1 separates the paper’s claims into proven, bounded-under-stated-assumptions, and empirical-or-heuristic; readers in a hurry may want to start there. Section 2 surveys EIP-1559, multi-resource fee market designs, and attestation-chain fee patterns as background. Section 3 fixes the system model: the schema as fee-market unit, validator income decomposition, demand profile taxonomy. Section 4 specifies the mechanism. Section 5 proves the cost-to-grind preservation theorem and analyzes four threat models. Section 6 analyzes incentives across three parties (validator, builder, sponsor). Section 7 walks through the Sovereign SDK integration and recommended v0 parameters. Section 8 positions per-schema fees against prior systems. Section 9 lists limitations and future work; Section 10 concludes; Section 11 collects frequently asked questions.

## 2. Background and Related Work

Fee market design has gone through three distinct generations on production chains: per-transaction auction (Bitcoin and pre-1559 Ethereum), unified base fee with target utilization (EIP-1559), and multi-resource fee markets (Solana, Aptos, Cosmos `x/feemarket`). This section surveys each and then positions attestation-chain fee patterns (EAS, Ceramic, EigenLayer AVS) against the per-schema thesis.

### 2.1 EIP-1559

Ethereum Improvement Proposal 1559 (Buterin et al., 2021) replaced Ethereum’s first-price gas auction with a base-fee + tip mechanism. Each block carries a base fee determined by the previous block’s utilization, with target utilization  $T = 0.5$  (half-full target). The per-block adjustment rule is:

$$b(t+1) = b(t) \cdot \left(1 + \xi \cdot \frac{u(t) - T}{T}\right), \quad \xi = \frac{1}{8}$$

allowing a maximum  $\pm 12.5\%$  swing per block. The base fee is burned (sent to a verifiably unspendable address); tips go to the proposer. The mechanism ships convergence properties around  $u = T$  that have held in production since August 2021.

**What EIP-1559 does well.** Base-fee burn anchors the chain’s monetary policy: high demand burns more, deflationary pressure aligns with usage. Target utilization gives predictable headroom (50% buffer over steady state, supporting normal-load latency guarantees). Per-block adjustment is bounded ( $\xi = 1/8$  prevents oscillation), and the mechanism is composable with sponsored-gas patterns at the application layer (ERC-4337 paymasters).

**What EIP-1559 cannot do that per-schema fees can.** A single base fee assumes a single workload. Ethereum’s heterogeneity (token transfers, DeFi, DA blobs since EIP-4844) has driven the chain toward separate fee markets for DA (blobspace) while keeping execution as one fee market. The trend is per-resource separation; this paper extends it to per-workload (schema) separation.

### 2.2 Multi-Resource Fee Markets

Three production chains have implemented variants of multi-dimensional pricing.

**Solana** charges a per-compute-unit (CU) fee with optional priority fees. The base unit price is set per-transaction by the submitter, subject to chain-enforced minimum and the proposer-tunable maximum CU per block. Priority fees compete for inclusion within the block. The mechanism is closer to a first-price auction than to EIP-1559’s adjusting base fee; convergence happens via signaling between users and proposers rather than via a deterministic curve.

**Cosmos `x/feemarket`** (cosmos-sdk v0.50+) provides an EIP-1559-inspired adjusting base fee at the chain level, with optional per-module fee multipliers. Each module (the `bank` module, the `staking` module, custom application modules) can declare a fee multiplier that scales the chain-wide base fee for transactions of that module type. This is the closest existing primitive to per-schema fees: per-module multipliers approximate per-workload pricing. The differences from this paper: (i) Cosmos multipliers are governance-set static parameters, not dynamic per-module utilization-driven, (ii) Cosmos has no per-module target-utilization mechanism, (iii) the burn-and-routing split is global, not per-module.

**Aptos gas pricing** uses a per-transaction gas-unit-price plus a max-gas-amount. The proposer admits transactions in tip-priority order. The mechanism is closer to a sealed-bid auction than to a smoothly-adjusting market. Aptos has no per-module differentiation; one pool of gas, one curve.

**EIP-4844 blob fees on Ethereum.** Independent of execution gas, blob transactions (data-availability blobs for L2s) have their own EIP-1559-style fee market. The blob base fee adjusts based on blob utilization, completely separate from execution-gas utilization. This is the first production example of *per-resource fee market separation* on a major chain. Per-schema fees extends the same idea from per-resource to per-workload granularity.

**What this paper takes from prior work.** The EIP-1559 adjustment formula. The Cosmos per-module multiplier intuition (per-module pricing matters). The EIP-4844 lesson that per-resource fee separation works in production. The Aptos / Solana lesson that simpler designs (per-transaction auctions) suffer at the heterogeneity edge.

**What this paper does that none do.** Make the schema (the workload type) the unit of fee adjustment, with full per-schema state (not just per-module multiplier), full per-schema target utilization, and a burn-versus-routing split that flows part of the non-burned base fee to the schema registrant.

## 2.3 Attestation Chains and Schema Heterogeneity

Three attestation-adjacent systems are worth comparing on the schema-heterogeneity axis.

**Ethereum Attestation Service (EAS)** is a smart-contract attestation framework on Ethereum. Schemas are first-class on-chain entities, but EAS inherits Ethereum’s unified EIP-1559 fee market: a single base fee plus tip applies to all EAS transactions regardless of schema. This is exactly the misalignment §1.3 describes. EAS’s design is reasonable for Ethereum (workloads are dominated by DeFi, not attestations), but a chain whose primary workload *is* attestations cannot afford to mix the fee surface.

**Ceramic** uses per-stream rentals (model fees) rather than per-attestation fees. Each Ceramic Model has a fee curve tied to the model’s storage footprint and update frequency. This is closer to a SaaS pricing model than a public-chain fee market: the per-model price is set by the model author and doesn’t adjust to demand. The intuition (per-workload pricing matters) is right; the mechanism (static fee curves) doesn’t fit chains that need dynamic price discovery.

**EigenLayer AVS economics.** Restaking AVSs (Actively Validated Services) have per-AVS slashing and reward economics: an AVS author specifies the rewards / slashing parameters for operators serving that AVS. This is a different axis (security economics, not transaction fees), but the analogy is structural: per-AVS economics is to restaking as per-schema fees is to attestation chains. Both treat the workload as the natural unit of economic differentiation.

**Ligate’s position.** Per-schema fees as a runtime primitive, with full per-schema dynamic adjustment (not static like Ceramic, not global like EAS, not security-economics-flavored like EigenLayer). The schema is the workload, and the workload is the fee-market unit. The §6 comparison table puts this paper alongside EIP-1559, Cosmos **x/feemarket**, Solana priority fees, and Aptos gas pricing across six axes; per-schema fees occupies a distinct design point.

### 3. System Model

#### 3.1 The Schema as Fee-Market Unit

A schema  $\sigma$  is a registered, named template on Ligate Chain. The PoUA paper (v0.8 §3.4) defines a schema as a tuple  $(\text{schema\_id}, \text{attestor\_set}, \text{validation\_rule}, \text{fee\_routing\_bps}, \text{metadata})$  where **schema\_id** is the canonical hash, **attestor\_set** is the threshold-signature group authorized to attest under it, and **validation\_rule** is the validity predicate. Each registered schema persists in chain state as a first-class entity. We extend the schema record with a **fee-market state**:

$$\text{FeeState}(\sigma) = (b_\sigma, u_\sigma(t), T_\sigma, \rho_\sigma, \tau_\sigma^{\min}, b_\sigma^{\min}, b_\sigma^{\max}, \xi_\sigma)$$

where:

- $b_\sigma \in [b_\sigma^{\min}, b_\sigma^{\max}]$  is the current per-schema base fee, denominated in the chain’s micro-unit (e.g., **uavow** once the rename in ligate-chain#457 lands).
- $u_\sigma(t) \in [0, 1]$  is the schema’s observed utilization at block  $t$ : the fraction of  $\sigma$ ’s allocated attestation slots filled in that block.
- $T_\sigma \in [0.1, 0.9]$  is the target utilization (§4.2 calibration table).
- $\rho_\sigma \in [0, 0.5]$  is the fee-routing fraction: the share of non-burned base fee that flows to the schema registrant. Default  $\rho_\sigma = 0$ .
- $\tau_\sigma^{\min} \geq 0$  is the tip floor: the minimum tip an attestation must offer to be admitted (§4.3).
- $b_\sigma^{\min}, b_\sigma^{\max}$  are governance-tunable clip bounds on the base fee.
- $\xi_\sigma \in (0, 1]$  is the per-block max-change rate (default  $\xi_\sigma = 1/8$ , matching EIP-1559).

**Why the schema is the natural unit.** Two arguments. First, **demand profiles differ by orders of magnitude per-schema** (§3.3). A single chain-wide base fee penalizes the low-demand schema with the high-demand schema’s congestion, and vice versa. The fee market should isolate demand signals at the same granularity that the chain isolates attestation work: per-schema.

Second, **the chain already maintains per-schema state for PoUA reputation accounting** (PoUA §4.3:  $g_v$  accumulates from valid attestation work, schema-tagged). Adding fee-market state per-schema is a natural extension of state that already exists. No new global-state surface; no new module boundary; the schema record gets four additional fields.

**State-tree cost.**  $\text{FeeState}(\sigma)$  adds approximately 64 bytes per schema (one u64 for  $b_\sigma$ , one for  $u_\sigma$  as a fixed-point fraction, eight u32 / u16 fields for the rest). At 1,000 registered schemas, this is  $\sim 64$  KB of state per epoch. Negligible against the chain’s expected state size at devnet + post-devnet scale.

**Registration cost.** Per PoUA §4.4, registering a new schema costs **RegisterSchema** fee plus the configured **RegisterAttestorSet** fee. The fee-market state is initialized at registration with  $(b_\sigma, u_\sigma, T_\sigma) = (b_\sigma^{\min}, 0, T_{\text{default}})$  and the registrant’s declared parameters for  $(\rho_\sigma, \tau_\sigma^{\min}, b_\sigma^{\min}, b_\sigma^{\max}, \xi_\sigma)$ . No additional registration cost is added by this paper; the existing **RegisterSchema** fee absorbs the new state.

#### 3.2 Validator Income Decomposition

A validator  $v$  proposing block  $B$  at slot  $t$  earns income from three streams. Let  $|B| = \{\alpha : \alpha \in B\}$  be the set of attestations included in  $B$ . Then:

$$R_v(B, t) = R_b + \sum_{\alpha \in B} \tau_\alpha + \sum_{\alpha \in B} (1 - \tau_{\text{burn}}) \cdot (1 - \rho_{\sigma(\alpha)}) \cdot b_{\sigma(\alpha)} \cdot |\alpha|$$

where:

- $R_b$  is the **protocol block reward** (chain-wide constant, set by governance; in PoUA v0 set as a small per-block emission until  $R_f$  stabilizes).
- $\tau_\alpha$  is the **tip** for attestation  $\alpha$ , flowing entirely to the proposer.
- $\sigma(\alpha)$  is the schema of attestation  $\alpha$ .
- $b_{\sigma(\alpha)}$  is the current per-schema base fee at slot  $t$  for  $\alpha$ 's schema.
- $|\alpha|$  is the size of the attestation in attestation-units (typically 1; see §3.3 for multi-unit attestations).
- $\tau_{\text{burn}}$  is the chain-wide burn fraction governed by PoUA §4.4.2's adaptive rebase. The complement  $(1 - \tau_{\text{burn}})$  is the non-burned share.
- $\rho_{\sigma(\alpha)}$  is the schema's fee-routing fraction. The complement  $(1 - \rho_{\sigma(\alpha)})$  is the validator's share of the non-burned amount.

The third term is the **validator's per-attestation base-fee share**, which depends on which schemas the validator includes. A validator who proposes blocks heavy in high-fee schemas earns more from this term; a validator who specializes in low-fee schemas earns less from this term but accumulates reputation just as fast (PoUA reputation update is fee-weighted by total fee, not validator-share-of-fee).

**Three rebate destinations per paid base fee.** Decomposing the base fee for one attestation  $\alpha$  of schema  $\sigma$ :

| Destination                 | Fraction of $b_\sigma$                             | Recipient                                    |
|-----------------------------|----------------------------------------------------|----------------------------------------------|
| Burn                        | $\tau_{\text{burn}}$                               | Pure-burn destination (PoUA §5.5.3)          |
| Schema registrant (routing) | $(1 - \tau_{\text{burn}}) \cdot \rho_\sigma$       | Schema's declared <b>fee_routing</b> address |
| Validator (proposer)        | $(1 - \tau_{\text{burn}}) \cdot (1 - \rho_\sigma)$ | Block proposer                               |

Tips  $\tau_\alpha$  are separate and flow 100% to the proposer.

**Why the validator's income is fee-schema-mixed.** A validator's per-block revenue depends on the schema mix in the block. This creates an incentive for the validator to prefer high-fee-schema attestations during proposal. PoUA §A.1's KL-divergence detector bounds this preference: the validator's schema-mix distribution is detected against the chain-wide null. A validator who deviates from the null distribution (e.g., consistently underweighting low-fee schemas) is flagged. The detector is the protocol-level enforcement that **fee-driven inclusion preferences cannot turn into censorship of low-fee schemas**.

§5.2 and §6.1 take this further: we show that under standard parameters, the §A.1 detector tolerance is wide enough to permit honest fee-driven preference but tight enough to flag adversarial cross-schema censorship.

### 3.3 Demand Profile Taxonomy

Attestation chains do not have one demand profile; they have several, and the per-schema design is justified by how different they are. We taxonomize three profiles and assign each a recommended target utilization.

**Profile A: High-volume, low-variance.** Steady-state attestation workloads with millions of attestations per day, smoothly distributed across time. Examples:

- **Themisra proof-of-prompt/v1** at maturity: every Claude / ChatGPT / Gemini session ends with an attestation submission. Demand correlates to user activity, smooth on multi-hour scales.
- **Content-authenticity attestations** for media pipelines: every uploaded video / image generates one attestation per processing step. Continuous load.

These profiles have low per-block variance and benefit from a base fee that prices steadily at the demand curve. Recommended  $T_\sigma = 0.5$  (default): balances predictability against revenue extraction. The 2x headroom over steady-state demand absorbs normal noise without triggering rapid base-fee swings.

**Profile B: Low-volume, high-value.** Rare attestations with high per-claim economic significance. Examples:

- **Sovereign-identity proofs:** government-issued identity assertions, attestor set is a small set of accredited identity providers. Demand is correlated to onboarding flows, hundreds per day at maturity.
- **Regulatory filings:** per-quarter or per-event compliance attestations. Sparse, predictable.

These profiles want to pack closer to demand peak (high  $T_\sigma$ ) because demand is predictable and the cost of paying slightly more for fast inclusion is low against the per-attestation value. Recommended  $T_\sigma = 0.7$ : 1.4x headroom over steady-state demand, with the higher utilization extracting revenue closer to the demand curve.

**Profile C: Bursty / event-driven.** Demand spikes 10x to 100x baseline for short windows around events. Examples:

- **NFT mint attestations** around drops: a single project may generate tens of thousands of attestations in a five-minute window, then return to baseline.
- **News-event-driven content provenance:** breaking news / live events / political moments generate concentrated bursts of **themisra.content-provenance/v1** attestations.
- **Iris-agent campaigns:** an Iris-hosted agent running a coordinated multi-hour campaign generates a burst of attestations across multiple schemas.

These profiles need heavy headroom. A spike that triggers base-fee climbing punishes legitimate users who happen to participate in the spike. Recommended  $T_\sigma = 0.3$ : 3.3x headroom over steady-state demand, prioritizing predictability for users during the burst over revenue extraction.

**Latency-critical addendum.** Schemas with hard latency requirements (e.g., Iris agent actions where the user is waiting on a sub-second response) want  $T_\sigma = 0.3$  even at low expected volume, to maintain immediate-inclusion guarantees during normal load. Latency is treated as a special case of bursty (the “burst” is occasional unexpected load).

**The taxonomy in one table:**

| Profile                            | Example                      | Recommended $T_\sigma$ | Recommended $\rho_\sigma$ |
|------------------------------------|------------------------------|------------------------|---------------------------|
| A: High-volume,<br>low-variance    | Themisra prompts             | 0.5                    | 0.1-0.3                   |
| B: Low-volume,<br>high-value       | Sovereign identity           | 0.7                    | 0.3-0.5                   |
| C: Bursty /<br>event-driven        | NFT mints, Iris<br>campaigns | 0.3                    | 0.1-0.3                   |
| Latency-critical<br>(special case) | Iris agent actions           | 0.3                    | 0-0.1                     |

These are architectural defaults. Devnet observation in v0.2.x will refine per-schema targets; schemas declare a preferred  $T_\sigma$  at registration; the chain enforces the protocol-level  $[0.1, 0.9]$  bound. The recommendation table is governance guidance, not a hard constraint.

**Attestation-unit granularity.** Within a schema, attestations may carry weight  $|\alpha| > 1$  for compound claims (a single attestation that references  $k$  sub-claims). The fee mechanism treats  $|\alpha|$  as a multiplier on base fee and tip: an attestation of size  $|\alpha| = 3$  pays  $3 \cdot b_\sigma + \tau_\alpha$  (the tip is per-attestation, not per-unit). This matches the rest of the chain’s resource-accounting convention.

## 4. Per-Schema Fee Mechanism

This section specifies the per-schema fee market: how the base fee adjusts in response to schema-specific demand, how priority within a schema is auctioned via tips, how base-fee revenue is split between burn and validator reward, and how the mechanism integrates with PoUA reputation accounting.

The mechanism is a per-schema instantiation of EIP-1559’s adjustment dynamics, with three modifications appropriate to attestation-native chains: (1) per-schema state rather than global state, (2) integration with PoUA’s adaptive  $\tau_{\text{burn}}$  rebase from PoUA §4.4.2, and (3) explicit accounting for sponsored-gas relayers (used by the Iris MCP relayer, design-partner-validated).

### 4.1 Base-Fee Adjustment Formula

For each registered schema  $\sigma \in \Sigma$ , the chain maintains a per-schema base fee  $b_\sigma(t)$ , target utilization  $T_\sigma$ , and observed utilization  $u_\sigma(t)$ . Per-block adjustment follows the EIP-1559 form:

$$b_\sigma(t+1) = b_\sigma(t) \cdot \left( 1 + \xi \cdot \frac{u_\sigma(t) - T_\sigma}{T_\sigma} \right)$$

clipped to  $[b_\sigma^{\min}, b_\sigma^{\max}]$ , where:

- $u_\sigma(t) \in [0, 1]$  is the fraction of schema  $\sigma$ ’s allocated attestation slots filled in block  $t$
- $T_\sigma \in (0, 1)$  is the schema’s target utilization (§4.2)
- $\xi$  is the per-block max-change parameter
- $[b_\sigma^{\min}, b_\sigma^{\max}]$  are governance-tunable clip bounds

**Recommended  $\xi$ .**  $\xi = 1/8$  matches EIP-1559’s calibration (max  $\pm 12.5\%$  per block). At the v0 block time of 12 seconds, this gives a  $\sim 64\%$  swing potential per epoch (6 blocks of 1.125); aggressive enough to track demand shifts, gentle enough to avoid oscillation.

**Allocated slots per schema.** A block’s total attestation capacity is partitioned across schemas at proposer-selection time. The simplest allocation is fixed-share-per-schema (each schema gets  $\lfloor C_{\text{block}} \cdot w_{\sigma} \rfloor$  slots,  $\sum w_{\sigma} = 1$ ); a richer allocation is governance-tunable per-schema utilization-aware. v0 ships fixed-share as the default.

**Convergence and stability.** The dynamics admit a fixed point at  $u_{\sigma}^* = T_{\sigma}$ . Around this fixed point, perturbations decay geometrically with rate  $1 - \xi$  at first order. The mechanism inherits EIP-1559’s convergence behavior; per-schema isolation means a high-utilization schema cannot drag down the base fee of a low-utilization schema. This is the central architectural advantage of the per-schema design.

**Drift interaction with PoUA  $\tau_{\text{burn}}$  rebase.** Both this paper’s per-schema base fee and PoUA §4.4.2’s  $\tau_{\text{burn}}$  are time-varying parameters. The composition is hierarchical:  $\tau_{\text{burn}}$  governs what fraction of  $b_{\sigma}$  is burned (§4.4);  $b_{\sigma}$  itself adjusts per-schema. The v0.8 PoUA paper §4.4.3 spec at `papers/poua/specs/eta-lambda-rebase.md` §5.2 verifies that the two rebases are first-order independent: schema-fee drift is the input to  $b_{\sigma}$ , while cost-to-grind drift is the input to  $\tau_{\text{burn}}$ .

## 4.2 Target Utilization Calibration

The target utilization  $T_{\sigma}$  controls the trade-off between **predictability** (low  $T_{\sigma}$ , more headroom for spikes) and **revenue extraction** (high  $T_{\sigma}$ , prices closer to demand peak).

**Recommended default:**  $T_{\sigma} = 0.5$ . Matches EIP-1559’s choice. Provides  $2\times$  headroom over steady-state demand.

### Schema-specific deviations.

| Schema profile                                             | Recommended $T_{\sigma}$ | Rationale                                                                     |
|------------------------------------------------------------|--------------------------|-------------------------------------------------------------------------------|
| Stable, low-volume (e.g., regulatory filings)              | 0.7                      | Variance is low; can pack closer to demand peak without congestion            |
| High-volume, low-variance (e.g., Themisra prompts)         | 0.5                      | Default; balances predictability and revenue                                  |
| Bursty (e.g., schema-bound-token mint events around drops) | 0.3                      | Heavy headroom to absorb spikes without triggering rapid base-fee climbs      |
| Real-time / time-critical (e.g., Iris agent actions)       | 0.3                      | Latency sensitivity demands immediate-inclusion guarantees during normal load |

[**Measured at v0.2.5+:** these are architectural defaults; devnet observation will refine per-schema targets. Schemas can declare a preferred  $T_{\sigma}$  at registration; the chain enforces governance bounds.]

**Bounds.**  $T_{\sigma}$  is bounded to  $[0.1, 0.9]$  at the protocol level. A schema requesting  $T_{\sigma} < 0.1$  is asking for  $> 10\times$  headroom over peak demand, which is wasteful; a schema requesting  $T_{\sigma} > 0.9$  leaves

no buffer for normal variance and would oscillate. Governance can adjust the bound but not the rate-limit (max one bound change per quarter).

### 4.3 Tip Mechanism

Within a schema’s allocated capacity, attestations are admitted in order of tip-per-attestation  $\tau_\alpha$ . Tips go to the proposing validator and are not subject to the base-fee burn (§4.4). The mechanism is identical in structure to EIP-1559’s priority fee; per-schema partitioning means tips compete only against same-schema attestations.

**Tip floor.** Schemas may declare a minimum tip  $\tau_\sigma^{\min}$  at registration to prevent zero-fee spam. Default  $\tau_\sigma^{\min} = 0$  (open to economic-signaling-only).

**Tip auction within block.** The proposer admits attestations greedily by descending  $\tau_\alpha$  until the schema’s allocated capacity is filled. Ties broken by first-seen mempool ordering. The greedy auction is welfare-suboptimal compared to a sealed-bid second-price design but matches EIP-1559’s actual deployment behavior; v2 extensions can revisit.

**Sponsored gas (paymaster pattern).** Iris-style relayers pay tips on behalf of agents that don’t hold \$AVOW. The protocol semantics are unchanged: a single signed transaction can declare a *fee payer* address distinct from the *attestor* address. The fee payer pays both base fee and tip; the attestor signs the attestation content. This composes orthogonally with native delegation (companion paper, v0.2): a delegated hot key can submit attestations whose fees are paid by a third-party paymaster. v2 of this paper details the formal fee-payer mechanism; v0.2 establishes that the design admits it cleanly.

### 4.4 Base-Fee Burn

Of every paid base fee  $b_\sigma \cdot |\alpha|$  for an admitted attestation  $\alpha$  of schema  $\sigma$ :

- A fraction  $\tau_{\text{burn}}$  is burned (sent to the pure-burn destination per PoUA §5.5.3)
- A fraction  $1 - \tau_{\text{burn}}$  is rebated to the schema’s fee-routing address (the registrant; up to 50% per OVERVIEW.md’s schema primitive)
- Tips  $\tau_\alpha$  are unaffected; they go to the proposing validator

**Why the schema gets the non-burned fraction.** The schema registrant pays the registration fee at schema-creation, including the cost of operating their attestor set. Routing a fraction of base fees to the registrant aligns incentive: schemas that drive volume earn proportional ongoing revenue, which funds attestor-set operations and incentivizes schema curation. This is the “build a chain on top of Ligate” path: a partner registers a schema, pays the registration fee, and earns ongoing fee revenue from that schema’s traffic.

**Per-schema fee-routing parameter.** At registration, a schema declares its fee-routing fraction  $\rho_\sigma \in [0, 0.5]$  (capped at 50% per protocol parameter). Effective burn is then:

$$\text{burn}(\sigma) = \tau_{\text{burn}} + (1 - \tau_{\text{burn}}) \cdot (1 - \rho_\sigma)$$

with the remainder  $(1 - \tau_{\text{burn}}) \cdot \rho_\sigma$  routed to the schema registrant.

**Interaction with PoUA  $\tau_{\text{burn}}$  rebase.**  $\tau_{\text{burn}}$  is the chain-wide burn fraction governed by PoUA §4.4.2’s adaptive rebase. The per-schema  $\rho_\sigma$  is set at registration and changes only via governance

proposal. The two parameters are independent:  $\tau_{\text{burn}}$  tracks economic-security drift,  $\rho_{\sigma}$  tracks schema-builder economics.

**Lemma 1 cost-to-grind preservation.** PoUA §5.5.3’s Lemma 1 bounds adversary cost-to-grind as  $F_{\text{net}} \geq \tau_{\text{burn}} \cdot \Delta r / (\eta \cdot \alpha_{\text{eff}})$ . The per-schema fee market does not weaken this bound: even if every schema sets  $\rho_{\sigma} = 0.5$ , the burned fraction per attestation is at least  $\tau_{\text{burn}} \cdot 0.5 \cdot b_{\sigma}$ , and the cost-to-grind floor scales with that burned amount. The paper’s central security claim survives the introduction of per-schema fee routing.

#### 4.5 Integration with PoUA

The per-schema fee market interacts with PoUA reputation in three places.

**Reputation accumulation is schema-agnostic.** Validators accumulate reputation  $g_v$  from valid attestation work across all schemas, weighted by fee paid (PoUA §4.3):

$$G_v^{\text{prop}}(t) = \sum_{B \in \text{Proposed}_v(t, t+E)} \sum_{\alpha \in B} \mathbb{1}[\alpha \text{ valid}] \cdot \text{fee}(\alpha)$$

where  $\text{fee}(\alpha) = b_{\sigma} + \tau_{\alpha}$  is the total fee paid (base + tip) for attestation  $\alpha$  of schema  $\sigma$ . A validator who proposes blocks heavy in high-fee schema attestations accumulates reputation faster than one who specializes in low-fee schemas, but both accumulate strictly more than zero. This preserves the §6.2 honest-equilibrium incentive structure.

**Validator income decomposition.** Per PoUA §6.1, validator income from block  $B$  at slot  $t$  is:

$$R_v(B, t) = R_b + \sum_{\alpha \in B} \tau_{\alpha} + \sum_{\alpha \in B} (1 - \tau_{\text{burn}}) \cdot (1 - \rho_{\sigma(\alpha)}) \cdot b_{\sigma(\alpha)}$$

where  $R_b$  is the protocol block reward,  $\tau_{\alpha}$  is the per-attestation tip, and the third term is the validator’s share of the per-schema base fee after burn and schema-routing. The validator’s revenue depends on schema mix, but the §A.1 KL-divergence detector enforces that schema mix tracks the chain-wide null distribution; a validator preferentially including high-fee-schema attestations gets flagged.

**Detector calibration accommodates per-schema fees.** PoUA §A.1 defines a KL-divergence detector against a chain-wide schema distribution null. With per-schema fees, validator-utility incentivizes schema-mix deviation; the detector calibration (see the `detectors.py` module under `prototypes/poua-sim`) tracks the actual per-schema arrival distribution. v0.2 of this paper specifies the joint calibration: detector null = empirical per-schema arrival distribution at the chain level, not at the validator level. v0.7.2 PoUA §A.1 already supports this; the per-schema fees mechanism does not require detector revision.

**Sponsored gas does not inflate reputation.** Iris-style relayers paying base fees and tips on behalf of agents do not accumulate reputation themselves (they are not validators). The agent submitting the attestation gets credited (reputation accumulates against the *attester*, not the *fee payer*); this matches the paper’s intent that reputation tracks productive work, not financial sponsorship.

## 5. Security Analysis

We analyze the per-schema fee market against five threat models. The first (§5.1) is the central theorem: PoUA’s cost-to-grind floor is preserved under per-schema isolation. The remaining four (§5.2-§5.5) analyze concrete cross-schema attack patterns and their defenses.

### 5.1 Cost-to-Grind Preservation Theorem

**Claim.** Under the per-schema fee market specified in §4, with arbitrary registered schema set  $\Sigma$  and per-schema routing fractions  $\{\rho_\sigma\}_{\sigma \in \Sigma}$  bounded by  $\rho_\sigma \in [0, 0.5]$ , PoUA’s §5.5.3 Lemma 1 cost-to-grind floor holds with the same constants:

$$F_{\text{net}}(\sigma) \geq \tau_{\text{burn}} \cdot \frac{\Delta r}{\eta \cdot \alpha_{\text{eff}}}$$

for every schema  $\sigma$ , where  $F_{\text{net}}(\sigma)$  is the non-recoverable cost an adversary pays per unit of reputation gained through attestations of schema  $\sigma$ .

**Setup.** PoUA Lemma 1 bounds the cost an adversary must pay to inflate their reputation by  $\Delta r$  through legitimate-looking attestation work. The bound assumes a per-attestation fee floor that is partly burned (non-recoverable to the adversary) and partly paid out (potentially recoverable if the adversary’s coalition includes both attestors and validators). The relevant constants:

- $\tau_{\text{burn}}$ : chain-wide burn fraction (PoUA §4.4.2, adaptively rebased)
- $\Delta r$ : target reputation gain
- $\eta$ : adversary’s coalition share of validator power
- $\alpha_{\text{eff}}$ : effective reputation-per-fee ratio under coalition’s attestation pattern

Lemma 1 establishes  $F_{\text{net}} \geq \tau_{\text{burn}} \cdot \Delta r / (\eta \cdot \alpha_{\text{eff}})$  in the chain-wide setting where every paid base fee contributes  $\tau_{\text{burn}}$  to burn and  $(1 - \tau_{\text{burn}})$  to validator income.

**The challenge of per-schema routing.** Under per-schema fees, a fraction  $(1 - \tau_{\text{burn}}) \cdot \rho_\sigma$  of every paid base fee flows to the schema registrant rather than to the validator. From an adversary’s perspective, this fraction is “potentially recoverable” if the adversary controls the schema registrant address (which they would, if they registered the schema being attacked). The naive concern: routing reduces the non-recoverable share, weakening the cost-to-grind floor.

**Proof of the bound.** The burned fraction per paid base fee is exactly  $\tau_{\text{burn}}$ , independent of  $\rho_\sigma$ . The routing fraction  $\rho_\sigma$  partitions the *non-burned* share between validator and schema registrant; it does not touch the burned share. Substituting into Lemma 1:

$$F_{\text{net}}(\sigma) = \tau_{\text{burn}} \cdot b_\sigma \cdot (\text{attestations submitted to gain } \Delta r)$$

The chain-wide formula  $F_{\text{net}} \geq \tau_{\text{burn}} \cdot \Delta r / (\eta \cdot \alpha_{\text{eff}})$  holds per-schema with the *same*  $\tau_{\text{burn}}$ . Per-schema base fees  $b_\sigma$  may differ from a hypothetical chain-wide base fee, but the proportionality between fee and reputation gained ( $\alpha_{\text{eff}}$ ) absorbs that difference. The adversary cannot reduce their cost-to-grind by registering a schema with their preferred  $\rho_\sigma$ : routing flows the *post-burn* amount, never the burned amount.

**Corollary (worst case  $\rho_\sigma = 0.5$  for every schema).** Even if every registered schema sets  $\rho_\sigma = 0.5$ , half of the non-burned base fee flows to the schema registrant. The burned fraction is still  $\tau_{\text{burn}}$  per attestation, the cost-to-grind floor is still  $F_{\text{net}} \geq \tau_{\text{burn}} \cdot \Delta r / (\eta \cdot \alpha_{\text{eff}})$ .

**Discussion.** The mechanism design choice that makes this work is keeping  $\tau_{\text{burn}}$  chain-wide rather than per-schema. A per-schema burn fraction  $\tau_{\text{burn},\sigma}$  would let the schema registrant trade burn for routing on their own schema, breaking the floor. The composition is: chain-wide  $\tau_{\text{burn}}$  (PoUA §4.4.2 adaptive, tracking economic-security drift), per-schema  $\rho_\sigma$  (set at registration, tracking schema-builder economics). The two parameters live at different layers of the protocol and do not interfere.

## 5.2 Cross-Schema Arbitrage by Validator Inclusion Preference

**Setup.** A validator’s per-block income (§3.2) depends on the schema mix in the block, because schemas have different  $b_\sigma$  and different  $\rho_\sigma$ . A profit-maximizing validator prefers to include attestations from schemas with the highest  $(1 - \tau_{\text{burn}}) \cdot (1 - \rho_\sigma) \cdot b_\sigma$  per unit of block capacity. Left unchecked, this preference could turn into censorship of low-fee schemas: a validator might consistently exclude low-fee attestations even when they offer competitive tips.

**Defense (per-schema slot allocation).** The mechanism (§4.1) allocates per-block attestation slots per schema,  $\lfloor C_{\text{block}} \cdot w_\sigma \rfloor$  with  $\sum w_\sigma = 1$ . A validator cannot fill an entire block with high-fee attestations from one schema; they must respect the per-schema slot allocation. This caps the magnitude of inclusion preference at the slot-allocation granularity.

**Defense (PoUA §A.1 KL-divergence detector).** Within the per-schema slot allocation, a validator might still prefer high-fee schemas at the margin. PoUA’s §A.1 detector tracks each validator’s empirical schema-mix distribution against the chain-wide null. A validator who consistently underweights low-fee schemas (beyond what the slot allocation permits) is flagged. The default tolerance from PoUA v0.7 §A.4 was calibrated against a unified fee market; per-schema fees do not require recalibration because the null distribution is the chain-wide observed arrival distribution, which already incorporates per-schema fee dynamics.

**Bound.** A validator deviating from the null by more than  $\delta$  in KL divergence triggers the §A.1 detector. From PoUA §A.4,  $\delta = 0.5$  at the v0 calibration. This permits substantial honest fee-driven preference (a validator can favor high-fee schemas at 2-3x the null weight without triggering) while flagging coordinated cross-schema censorship.

**Worst-case attack.** An adversary controlling enough validator power could selectively exclude one schema entirely (drop its slot-allocated bandwidth). This is the classic single-schema censorship attack; PoUA §5.2 already covers it (safety inheritance plus the force-include path in ligate-chain). Per-schema fees do not introduce a new censorship surface; they inherit PoUA’s existing defenses against single-schema censorship.

## 5.3 Base-Fee Manipulation by Validator Coalition

**Setup.** A validator coalition with control over a fraction  $\eta$  of proposer slots could selectively pump or suppress a target schema’s base fee. Including high-tip attestations from schema  $\sigma$  artificially inflates  $u_\sigma$ , climbing  $b_\sigma$ ; excluding attestations from  $\sigma$  artificially deflates  $u_\sigma$ , dropping  $b_\sigma$ . Either direction could be used to grief legitimate users of  $\sigma$  or to extract surplus value for the coalition.

**Bound (max-change-per-block).** The §4.1 adjustment formula clips per-block changes to  $\pm\xi$  (default  $\xi = 1/8$ , max 12.5% swing). A coalition can move  $b_\sigma$  by at most  $\xi$  per block they propose.

Even with  $\eta = 0.3$  (an adversary cartel), the expected swing per block is  $0.3 \cdot \xi = 3.75\%$ . Over an epoch of 6 blocks (12-second block time, 72-second epoch), the maximum cumulative swing is  $\sim 25\%$  from the coalition’s contribution. Bounded enough that legitimate users see a transient surge, not a permanent dislocation.

**Bound (mempool transparency).**  $b_\sigma$  adjustment is deterministic from observed utilization. An adversary cannot fabricate utilization out of thin air; they must actually include or exclude real attestations. Mempool observability means the manipulation is *visible* to honest validators: a sudden spike of high-tip attestations from one address pattern, included only by one validator cartel, is detectable by the §A.1 detector and by ad-hoc analysis of the mempool.

**Bound (economic counterweight from tips).** A coalition extracting surplus value through base-fee manipulation must absorb the cost of including high-tip attestations they themselves submit. The tip flows to *some* validator (the proposer); if the coalition is the proposer, they earn the tip back, but they also paid the base fee that gets burned. Net: the coalition pays  $\tau_{\text{burn}}$  to burn for every attestation they use to manipulate the base fee, just like any other adversary running on a PoUA chain.

## 5.4 Fee-Griefing Across Schemas

**Setup.** An adversary submits high-tip attestations to a target schema  $\sigma_{\text{target}}$  to inflate  $u_{\sigma_{\text{target}}}$  and drive up  $b_{\sigma_{\text{target}}}$ . Legitimate users of  $\sigma_{\text{target}}$  then face elevated fees. The attack cost is the adversary’s tips (paid to proposers); the attack benefit is making the target schema expensive for legitimate users.

**Defense (per-schema isolation).** This attack pattern is contained within  $\sigma_{\text{target}}$ . Other schemas’ base fees are unaffected. The blast radius of the attack is limited to the target schema’s users, not the entire chain. Compared to the same attack pattern under a unified fee market (where griefing one workload climbs the chain-wide base fee), per-schema fees *reduce* the attack’s blast radius.

**Defense (max-change rate-limit).** The same  $\xi = 1/8$  bound applies. The adversary cannot move  $b_{\sigma_{\text{target}}}$  faster than 12.5% per block. To sustain elevated  $b_{\sigma_{\text{target}}}$ , the adversary must keep submitting high-tip attestations indefinitely; the attack has linear cost in attack duration.

**Defense (cost-to-grief vs cost-to-grieved).** Each attack-attestation costs the adversary  $b_\sigma + \tau_\alpha$ . The legitimate user’s price increase from one attack-attestation is small (one increment of  $\xi/|B|$  in the next base fee). The cost ratio of attacker-spend to victim-cost-increase is on the order of  $|B|/\xi$  per attestation, which under v0 parameters ( $|B| \sim 100$ ,  $\xi = 1/8$ ) is about 800. Each \$1 the attacker spends raises legitimate-user cost by \$0.0012 per attestation. The attack scales poorly.

**Worst case.** A well-resourced adversary running sustained griefing against a low-volume schema could double its base fee for hours. The legitimate users of that schema either pay the elevated fee, wait it out, or migrate to a different schema. The attack is unpleasant but bounded; the chain does not lose money (most of the attacker spend is burned), and the target schema’s users have product-layer mitigations (longer time-bounds on grants, deferred submission, alternative schemas for the same use case).

## 5.5 Sponsored-Gas Adversarial Patterns

**Setup.** Iris pays base fees and tips on behalf of customer agents. The Iris pricing model is monthly USD subscription against expected per-attestation \$AVOW cost over the billing period. An

adversary running an Iris-customer agent could try to exhaust Iris’s budget, or could cause Iris’s pre-committed price curve to under-collect against actual costs.

**Pattern A: budget exhaustion.** Adversary agent submits attestation floods, consuming Iris’s budgeted gas faster than the subscription allows. Iris must either reject the agent (revoking the grant per native-delegation §4.2) or absorb the excess cost.

**Defense.** Iris’s monitoring layer rate-limits per-agent attestation volume against the customer’s subscription tier. Native delegation grants (companion paper §3.4) carry a time-bound; Iris can include a per-grant attestation cap as part of the application-layer scope predicate (extending the protocol-level scope, which only includes schema set and action set). The combination of per-grant attestation cap + per-billing-period subscription cap limits adversary-customer impact to the customer’s own subscription.

**Pattern B: base-fee surge exploitation.** Adversary agent submits a flood of attestations to drive up  $b_\sigma$ , then continues normal usage at the elevated fee, forcing Iris to pay more per attestation than the pre-committed price curve assumed. This is fee-griefing (§5.4) applied to a sponsor rather than a direct user.

**Defense.** Iris’s pricing model includes a base-fee surge buffer: subscription tiers price against the 90th-percentile observed  $b_\sigma$  over the prior 30-day window, with a clip if observed  $b_\sigma$  exceeds a threshold. If the observed fee climbs beyond the clip, Iris reserves the right to throttle non-subscription-paying volume (e.g., the adversary agent gets de-prioritized while legitimate customers continue at the pre-committed rate). The product-layer pricing discipline handles base-fee surge variance.

**Defense (in-protocol part).** The per-schema fee market’s max-change-per-block ( $\xi = 1/8$ ) caps how quickly an adversary can move  $b_\sigma$ . Iris’s pricing buffer needs to absorb at most  $\sim 12.5\% \times (\text{blocks per billing cycle})$  in worst-case fee climb. At 30-day billing and 12-second block time, that is the theoretical maximum; in practice, fees decay back to equilibrium once the attack stops, so the worst-case is bounded by the attack duration.

**Pattern C: routing-fraction exploitation.** An adversary registers a schema with  $\rho_\sigma = 0.5$  (the maximum) and induces high traffic. They collect  $(1 - \tau_{\text{burn}}) \cdot 0.5$  of every paid base fee. The “attack” here is just a successful schema launch with maximum routing; it is not really an attack, but it is worth noting that schema authors who set  $\rho_\sigma = 0.5$  are economically equivalent to chain validators in their cut of fee revenue.

**Mitigation.** None needed at protocol level; the  $\rho_\sigma \leq 0.5$  bound is governance-set and prevents schema authors from extracting more than half of non-burned fees. Beyond that, schema authors who choose high  $\rho_\sigma$  trade lower base-fee-validator-share for higher schema-revenue, which is a legitimate business choice. The cost-to-grind theorem (§5.1) ensures the chain’s security floor is unaffected.

---

## 6. Incentive Analysis

Section 5 established that the per-schema fee market does not weaken the chain’s protocol-level security (cost-to-grind floor preserved, cross-schema attacks bounded). This section turns to the *behavioral* layer: under the mechanism, when is honest participation the rational choice for each

party? We analyze three party types: validators (who include attestations), builders (where separated, who bundle and propose blocks), and sponsors (Iris-style relayers paying gas on behalf of agents). The PoUA §6 incentive analysis covers the validator-as-attestor relationship; this section focuses on the per-schema-fee-specific incentives.

## 6.1 Validator Incentive to Honor All Schemas

**Question.** Under the per-schema fee market, does a validator have any incentive to systematically prefer high-fee schemas in their inclusion choices, even at the cost of violating PoUA §A.1’s KL-divergence detector tolerance?

**Setup.** A validator’s per-block income (§3.2) depends on the schema mix in the block. Within slot allocation  $w_\sigma$  per schema, the validator can choose which attestations to include. The naive maximizer picks attestations with the highest  $b_\sigma \cdot (1 - \rho_\sigma) \cdot (1 - \tau_{\text{burn}}) + \tau_\alpha$  per slot. This biases toward high-fee, low-routing schemas.

**Answer.** No, not in equilibrium. Two arguments.

First, the **PoUA §A.1 detector** flags persistent schema-mix deviation from the chain-wide null. A validator who consistently underweights low-fee schemas (beyond the slot-allocation cap and beyond honest fee-driven preference) triggers the detector. The penalty (per PoUA §A.4) is a reputation slash; the magnitude is calibrated to make sustained deviation strictly negative-EV against expected fee gain. From §5.2, the v0 tolerance permits  $\sim 3\times$  null-weight headroom, which is wider than honest fee-driven preference under typical schema distributions. A rational validator stays within tolerance.

Second, **PoUA reputation accumulates schema-agnostically**. A validator who specializes in high-fee schemas accumulates reputation no faster than one who serves the full schema distribution (PoUA §4.3 reputation update is fee-weighted by total fee, not validator-share-of-fee). The validator’s reputation, which is a forward-revenue stream (PoUA §6.3), grows at the same rate regardless of inclusion preference. Selecting high-fee schemas trades transient per-block income against detector risk without compounding reputation upside.

**Equilibrium.** Validators include attestations from all schemas roughly in proportion to the chain-wide arrival distribution (the §A.1 null). Honest fee-driven preference within tolerance is permitted and expected; systematic cross-schema censorship is dominated by honest behavior.

**Sensitivity to slot allocation.** The per-block slot allocation  $w_\sigma$  determines how much room validators have for inclusion preference. With strict per-schema slot caps, the validator has no choice within a schema’s slots; with loose caps (or no caps), the validator has more freedom. v0 ships strict caps (each schema gets exactly its allocated slots, no overflow), which makes the §A.1 detector mostly redundant for cross-schema preference. Future relaxation is governance-tunable.

## 6.2 Builder Incentive (Where Proposer and Builder Are Separated)

**Question.** In an architecture where block proposing and block building are separated (e.g., MEV-Boost on Ethereum, where proposers receive blocks from external builders), how do builders order attestations across schemas, and does this affect per-schema base-fee dynamics?

**Setup.** A builder constructs a block to maximize the value extractable from MEV (sandwiching, arbitrage, sniping) and from tips. They submit the block to a proposer in exchange for a bid. The proposer accepts the highest bid. Under per-schema fees, the builder’s optimization includes

schema mix: which attestations to include, in what order, to maximize tips + MEV - paid base fees.

**v0 architecture.** Ligate Chain v0 does not separate proposer and builder; the proposer constructs their own blocks. Builder incentives are not directly relevant. The §6.1 analysis covers the unified-actor case.

**Future architectures (v1+).** When (or if) Ligate Chain adopts a separated-proposer-builder architecture, three considerations arise. (1) The builder’s incentive to censor attestations from low-fee schemas is the same as the validator’s; the §A.1 detector mechanism extends to the builder by tracking builder-submitted blocks’ schema mix against the null. (2) MEV opportunities specific to attestation chains (e.g., attestation reordering for time-sensitive proofs, schema-spanning bundles) are not well-studied; a separate paper would treat them. (3) Builder bids interact with per-schema base fees: a builder bidding for inclusion of high-fee attestations effectively pays the validator a portion of the base fee that would otherwise burn. This is a slow leak in the cost-to-grind floor (the burned fraction shrinks by the builder’s share); §5.1’s theorem assumes proposer = builder and would need adjustment under separation.

We defer detailed treatment of MEV and separated-builder architectures to a follow-up paper; v0.2 establishes that the mechanism is well-defined in the unified-actor architecture Ligate Chain v0 ships.

### 6.3 Sponsor Incentive (Iris and Other Relayers)

**Question.** When is sponsored gas (a third party paying base fee and tip on behalf of an attestor) economically rational for the sponsor under per-schema fees?

**Setup.** Iris pays gas for autonomous agents under bounded-time grants (companion paper §3.4). Iris bills the user in USD monthly. The Iris pricing model needs three things to work: (i) the USD-per-month covers the expected per-attestation \$AVOW cost over the billing period plus operating margin; (ii) the variance in per-attestation cost is small enough that Iris’s margin absorbs typical fluctuation; (iii) the worst-case fee surge has a clip mechanism so Iris is not exposed to unbounded loss.

**Per-schema fees make Iris’s pricing model better, not worse.** Under a unified fee market, Iris’s per-attestation cost varies with chain-wide congestion, which is volatile and uncorrelated with the user’s behavior. Under per-schema fees, Iris’s per-attestation cost varies with the schema the user is attesting under, which is *correlated* with the user’s behavior (the user picks the schema; the fee tracks demand on that schema). Iris can offer per-schema pricing tiers: Themisra-only subscription at \$X/month, multi-schema at \$Y/month, identity-proof passthrough at \$Z/month.

**Sponsor margin model.**

$$\pi_{\text{Iris}} = R_{\text{sub}} - \mathbb{E} \left[ \sum_{\alpha \in \text{user's attestations}} (b_{\sigma(\alpha)} + \tau_{\alpha}) \right] - C_{\text{ops}}$$

where  $R_{\text{sub}}$  is monthly subscription revenue, the expectation is over attestation volume and per-schema base-fee dynamics, and  $C_{\text{ops}}$  is Iris’s infrastructure cost. Per-schema fees let Iris partition this expectation per-schema, hedge each component separately, and price subscription tiers accordingly.

**Pre-committed fee curve and surge buffer.** Iris’s subscription pricing locks the customer to a \$/month rate. The implied per-attestation rate is fixed at subscription start. If  $b_\sigma$  surges during the billing period (e.g., a Themisra spike), Iris absorbs the cost. The §5.5 analysis bounded the worst-case surge cost; in practice, Iris prices the buffer at  $\sim 25\%$  above the 90th-percentile observed  $b_\sigma$  over the prior 30-day window. The buffer is the margin Iris collects; it eats some buffer during surges and accumulates the rest as profit.

**Equilibrium.** Iris’s economic existence depends on the buffer being large enough to absorb realistic surges. Per-schema isolation reduces the magnitude of surges relative to a unified fee market (surges are bounded to one schema, not chain-wide), which improves the economics of the buffer. Iris is therefore *more viable* on a per-schema fee chain than on a unified-fee chain at the same chain-wide demand level.

**Alternative sponsorship models.** Beyond Iris, other sponsor models could exist: pay-per-attestation (no subscription, sponsor pays per-event with possible client-side limit), retroactive reimbursement (user pays first, sponsor reimburses based on usage tier), schema-author sponsorship (the schema author subsidizes their own schema’s fees to bootstrap adoption). Each has different incentive properties under per-schema fees; v0.2 documents Iris’s subscription model as the canonical case and notes that the mechanism does not preclude alternatives.

#### 6.4 Schema-Author Incentive to Set $\rho_\sigma$ Correctly

**Question.** When does a schema author set  $\rho_\sigma$  to maximize their schema’s success?

**Setup.**  $\rho_\sigma \in [0, 0.5]$  is set at schema registration. Higher  $\rho_\sigma$  means more fee revenue flows to the registrant (the schema author); lower  $\rho_\sigma$  means more flows to validators including the attestations. The registrant’s income from  $\sigma$  is  $(1 - \tau_{\text{burn}}) \cdot \rho_\sigma \cdot \mathbb{E}[\text{volume} \cdot b_\sigma]$ .

**Trade-off.** High  $\rho_\sigma$  extracts more per-attestation revenue but disincentivizes validators from including the schema’s attestations (their take per attestation is lower; they prefer schemas with lower  $\rho_\sigma$  given equal  $b_\sigma$ ). Low  $\rho_\sigma$  gives validators stronger inclusion preference but leaves money on the table for the schema author.

**Equilibrium.** Schema authors set  $\rho_\sigma$  to balance per-attestation extraction against expected volume. For high-volume low-margin schemas (Themisra), the optimal  $\rho_\sigma$  is modest (0.1-0.3): the schema author wants volume more than per-attestation share, and high  $\rho_\sigma$  would slow inclusion. For low-volume high-value schemas (sovereign identity), the optimal  $\rho_\sigma$  is higher (0.3-0.5): the schema author can afford to extract more because validators have less leverage over low-volume schemas (the slot allocation is small; inclusion preference matters less).

The §4.2 calibration table makes this recommendation explicit. Schema authors who deviate from the recommendation are not penalized at the protocol level (within the  $[0, 0.5]$  bound), but the recommendation is governance guidance based on the trade-off above.

#### 6.5 Equilibrium Summary

Across the four parties:

- **Validators** include attestations from all schemas roughly in proportion to the chain-wide null, with honest fee-driven preference within PoUA §A.1 tolerance. Systematic cross-schema censorship is dominated.

- **Builders** (where separated) inherit the same incentive structure as validators in v0; the §A.1 detector extends to builder-submitted blocks. v1+ architectures may need a separate paper.
- **Sponsors** (Iris) find sponsored gas economically rational on per-schema fees because per-schema isolation reduces fee variance compared to a unified fee market. Iris’s pricing model improves under this paper’s mechanism.
- **Schema authors** set  $\rho_\sigma$  to balance per-attestation extraction against volume, with the §4.2 calibration table providing governance guidance.

The §5 security theorems ensure that no party can extract value by violating the protocol; this section verifies that no party gains by deviating from the recommended behavior. The Nash equilibrium is honest fee-driven participation with reputation-aware inclusion, sustained by the asymmetric incentive structures above.

---

## 7. Implementation in Ligate Chain

### 7.1 Sovereign SDK Integration Points

Per-schema fees integrate with Ligate Chain’s Sovereign SDK rollup at three specific extension points. We describe each as it would land in the `ligate-chain` repository.

**Extension 1: schema record carries fee-market state.** The schema record in the attestation crate’s `state.rs` (or equivalent) extends to hold a `FeeMarketState` struct with the fields enumerated in §3.1 (`base_fee`, `observed_utilization`, `target_utilization`, `routing_fraction`, `tip_floor`, `fee_min`, `fee_max`, `adjustment_rate`). The state is initialized at `RegisterSchema` time with registrant-declared values subject to protocol bounds. State-tree update on registration is unchanged in shape, just larger payload.

**Extension 2: per-block base-fee update hook.** A new state-transition hook runs at end-of-block, after attestation admission and before block finalization. The hook iterates over all schemas that had attestations in the block, computes the observed utilization  $u_\sigma$ , and applies the §4.1 adjustment formula. The hook is deterministic from block contents and the prior state; light-client verification is straightforward. Implementation: ~80 lines of Rust in a new `fee_market.rs` module under the attestation crate.

**Extension 3: tip accounting in proposer income.** The block proposer’s reward calculation extends to sum per-attestation tips and per-attestation base-fee shares. The accounting hook runs in the same block-finalization sweep, accumulating into the proposer’s bank-module balance. Implementation: ~40 lines, modifying the existing reward-accumulation path.

**State-tree cost.** ~64B per schema (§3.1). At 1,000 registered schemas, ~ 64 KB. Negligible.

**Compute cost per block.**  $O(|\Sigma_{\text{block}}|)$  where  $|\Sigma_{\text{block}}|$  is the number of distinct schemas in the block. At a v0 cap of ~ 50 schemas per block, the per-block compute is dominated by the per-attestation work; the fee-market update is a small constant multiple of schema count.

**Light-client verification.** A light client wanting to verify that  $b_\sigma$  at block  $t$  is consistent with chain state reads the schema’s `FeeMarketState` (one state-tree lookup) and the prior block’s state (one more lookup), then evaluates the §4.1 adjustment formula locally.  $O(1)$  verification per schema per block.

## 7.2 Recommended v0 Parameters

The v0 launch parameters, subject to governance adjustment as devnet data refines them:

| Parameter                              | Default                | Range       | Source                 |
|----------------------------------------|------------------------|-------------|------------------------|
| Adjustment rate $\xi$                  | 1/8                    | [1/16, 1/4] | §4.1, matches EIP-1559 |
| Target utilization $T_\sigma$ default  | 0.5                    | [0.1, 0.9]  | §4.2, matches EIP-1559 |
| Routing fraction $\rho_\sigma$ default | 0                      | [0, 0.5]    | §4.4                   |
| Tip floor $\tau_\sigma^{\min}$ default | 0                      | $\geq 0$    | §4.3, open by default  |
| Base-fee min $b_\sigma^{\min}$         | 1 uavow                | governance  | §4.1                   |
| Base-fee max $b_\sigma^{\max}$         | $10^{12}$ uavow        | governance  | §4.1                   |
| Slot allocation $w_\sigma$             | proportional to volume | governance  | §4.1                   |

Schema authors declare per-schema overrides at registration within these bounds.

**Profile-aware defaults.** Per §4.2 calibration table, the chain offers three pre-configured target-utilization profiles at registration: **high-volume** ( $T_\sigma = 0.5, \rho_\sigma = 0.2$ ), **high-value** ( $T_\sigma = 0.7, \rho_\sigma = 0.4$ ), and **bursty** ( $T_\sigma = 0.3, \rho_\sigma = 0.1$ ). The schema author picks one or specifies custom parameters within bounds.

## 7.3 Migration from Unified Fee Market

Ligate Chain v0 ships per-schema fees from genesis. There is no migration from a unified fee market because the chain has not had a unified fee market at any prior phase. The transition is internal-only: the protocol designs the fee market as per-schema from the start.

For chains that adopt this mechanism after running a unified fee market, a governance-mediated migration would (1) introduce per-schema state initialized at the chain-wide base fee for every schema, (2) freeze the global base-fee adjustment for one epoch while per-schema adjustments warm up, (3) cut over to per-schema after warmup. This is a known pattern from EIP-1559’s introduction on Ethereum. Detailed migration mechanics are out of scope for v0.2 since Ligate Chain does not need them.

## 7.4 Test Vectors and Reference Simulator

A reference simulator under `prototypes/per-schema-fees-sim/` (planned milestone M1, after this paper lands) will provide cross-language test vectors for the §4.1 base-fee adjustment formula at canonical inputs. The simulator follows the v0.7-PoUA discipline: every numerical claim in this paper that involves per-schema dynamics ( $\xi = 1/8$  convergence rate,  $\rho_\sigma \leq 0.5$  cost-to-grind preservation, §5.4 attack-cost ratio of  $\sim 800$ ) gets a corresponding simulator test.

Test vectors are written in the same JSON format as `prototypes/poua-sim/test_vectors/`: input `(b_t, u_t, T_sigma, xi)` produces expected output `b_{t+1}` to a fixed precision. A Rust imple-

mentation in `ligate-chain` re-validates the same algebra at a different precision and language, catching subtle floating-point drift.

---

## 7. Implementation in Ligate Chain

### 7.1 Sovereign SDK Integration Points

[**v0.2:** Per-schema state extension: new `FeeMarketState` field on schemas. Updated runtime hook for base-fee adjustment at epoch boundaries.]

### 7.2 Recommended v0 Parameters

[**v0.2:** Per-schema  $T_\sigma$ ,  $\xi$ ,  $b_\sigma^{\min}$ ,  $b_\sigma^{\max}$  defaults.]

### 7.3 Migration from Unified Fee Market

[**v0.2:** Bootstrap-period unified fee market; activation of per-schema markets via governance after warmup.]

---

## 8. Comparison with Prior Systems

Per-schema fees occupies a distinct point in the design space of blockchain fee markets. The §2 background surveyed the related systems; this section places them in a single table for direct comparison.

| Axis                                          | EIP-1559<br>(Ethereum)                    | Solana priority<br>fees                | Cosmos<br>x/feemarket                         | Aptos gas<br>pricing          | Per-schema fees<br>(this paper)                                                                                      |
|-----------------------------------------------|-------------------------------------------|----------------------------------------|-----------------------------------------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Pricing<br/>granularity</b>                | Chain-wide (single<br>base fee)           | Per-transaction<br>(user-set)          | Per-module<br>multiplier<br>(chain-wide base) | Per-transaction<br>(user-set) | Per-schema (full state<br>per schema)                                                                                |
| <b>Target<br/>utilization<br/>mechanism</b>   | Yes ( $T = 0.5$ )                         | No (auction)                           | Yes (chain-wide $T$ )                         | No (auction)                  | Yes, per-schema ( $T_\sigma$ at<br>registration)                                                                     |
| <b>Dynamic<br/>adjustment</b>                 | Yes ( $\xi = 1/8$ )                       | No (user-set per<br>tx)                | Yes (chain-wide $\xi$ )                       | No (user-set per<br>tx)       | Yes, per-schema ( $\xi_\sigma$ )                                                                                     |
| <b>Burn semantics</b>                         | Base fee burned<br>100%                   | N/A (no burn)                          | Base fee burned<br>100%                       | N/A                           | $\tau_{\text{burn}}$ chain-wide;<br>( $1 - \tau_{\text{burn}}$ ) split<br>between validator and<br>schema registrant |
| <b>Workload-<br/>author revenue<br/>share</b> | None                                      | None                                   | None                                          | None                          | $\rho_\sigma \in [0, 0.5]$ at schema<br>registration                                                                 |
| <b>Sponsor-<br/>friendly<br/>composition</b>  | ERC-4337<br>paymaster (contract<br>layer) | Fee-payer field<br>(transaction layer) | Module-layer<br>extension                     | None native                   | Native (fee-payer field,<br>composes with<br>delegation)                                                             |
| <b>Multi-workload<br/>accommodation</b>       | Poor (unified)                            | Poor (auction)                         | Moderate<br>(per-module<br>multiplier)        | Poor (unified)                | Native (per-schema<br>isolation)                                                                                     |
| <b>Cost-to-grind<br/>preservation</b>         | Yes (Ethereum-side<br>argument)           | N/A                                    | Yes (Cosmos-side<br>argument)                 | N/A                           | Yes (§5.1 theorem<br>under per-schema<br>isolation)                                                                  |
| <b>Production<br/>status</b>                  | Live since Aug 2021                       | Live                                   | Live (cosmos-sdk<br>v0.50+)                   | Live                          | Specification                                                                                                        |

The comparators each solve a different subset of the fee-market design problem. **EIP-1559** maximizes simplicity at the cost of multi-workload accommodation; it works on a single-workload chain (Ethereum’s pre-blob era) and starts to break under multi-workload pressure (EIP-4844 blobs needed a separate fee market). **Solana priority fees** and **Aptos gas pricing** both rely on per-transaction auctions, which work mechanically but provide no price discovery for low-volume workloads. **Cosmos x/feemarket** comes closest to per-schema fees in shape: per-module multipliers approximate per-workload pricing. The differences are that Cosmos multipliers are governance-set static parameters (not dynamic per-module utilization-driven), Cosmos has no per-module target utilization (so the workloads share an adjustment signal), and the burn-and-routing split is global.

**Per-schema fees** occupies the unique design point of full per-workload state with dynamic per-workload adjustment, plus a configurable workload-author revenue share that does not weaken the cost-to-grind floor. The §3.1 argument that the schema is the natural fee-market unit, combined with the §5.1 theorem that PoUA’s security floor is preserved, gives this mechanism a clean place in the design space that no prior system occupies.

The choice to drop arbitrary per-resource pricing (gas / storage / DA as separate axes) is deliberate. Per-schema fees treats workload-type as the composite resource, which is the dimension that matters for attestation-native chains. Future work can extend the mechanism to multi-resource within a schema; v0.2 ships the schema as the primary axis.



## 9. Limitations and Future Work

The v0.2 mechanism specifies per-schema EIP-1559 dynamics on a single-chain, single-workload-type axis. Four extensions to that surface remain explicitly out of scope; we document each here.

### 9.1 Empirical Calibration Pending Devnet

The §4.2 calibration table is an architectural default informed by EIP-1559’s production track record and the demand-profile taxonomy in §3.3. Real per-schema demand curves on Ligate Chain will not be observable until devnet operation produces data. v0.2.x updates will refine the recommended  $T_\sigma$  values per profile based on empirical observation. The protocol-level bounds  $T_\sigma \in [0.1, 0.9]$  and  $\rho_\sigma \in [0, 0.5]$  are conservative enough that the defaults are unlikely to need expansion, but the within-bound recommendations will be tuned.

### 9.2 Sponsored-Gas Adversarial Model is Heuristic

The §5.5 analysis of sponsored-gas adversarial patterns assumes a specific Iris pricing model (monthly USD subscription, surge buffer at 25% above 90th-percentile prior-window observed fee). Other sponsorship models (per-attestation pay-as-you-go, retroactive reimbursement, schema-author-funded subsidies) have different threat surfaces. v0.3 should formalize a general sponsorship model and prove threat-model bounds for each.

### 9.3 Multi-Resource Fee Markets Within a Schema

The mechanism prices the schema as a single composite resource. Within a schema, sub-resources (compute, storage, DA bandwidth, validator attention) all share the schema’s  $b_\sigma$ . For schemas with

markedly different sub-resource demand (e.g., a Themisra prompt is compute-light but DA-heavy if the attestation includes a hash of the prompt content), this aggregation is imperfect. v0.4 could introduce per-sub-resource pricing within a schema, layered on top of the per-schema state. The mechanism design here is non-trivial because each schema’s sub-resource weights would need their own adjustment dynamics; a separate paper.

#### 9.4 Cross-Chain Fee-Market Portability

If a schema registered on Ligate Chain is also recognized on a counterparty chain via IBC (or any other cross-chain identity protocol), the counterparty chain needs price discovery for the schema’s attestations on its own block space. The current paper assumes single-chain operation. v0.3 or v0.4 could specify the cross-chain fee-market portability, leveraging IBC light-client proofs of  $b_\sigma$  on the source chain. The complications are (i) IBC update latency means cross-chain fee data is stale by the IBC round-trip, (ii) the counterparty chain’s slot allocation may not match Ligate’s, (iii) cost-to-grind preservation needs re-verification when slashing crosses chains. Each is a separable problem; together they constitute a follow-on paper.

#### 9.5 Separated-Proposer-Builder Architectures

§6.2 noted that the v0 unified-actor architecture (proposer = builder) simplifies the analysis. v1+ architectures with separated proposer and builder (MEV-Boost-style) would need (i) extension of PoUA §A.1 detector to builder-submitted blocks, (ii) detailed analysis of MEV opportunities specific to attestation chains, (iii) revision of the §5.1 cost-to-grind theorem if builder bids partially capture the burn share. The v0.2 mechanism is well-defined under unified-actor; separated-architecture analysis is future work.

---

## 10. Conclusion

Per-schema fee markets are the natural unit for attestation-native chains. The chain already accounts per-schema for attestation work (PoUA reputation update is schema-tagged); the fee market should track that granularity. The §1.3 misalignment problem is the central motivation: a unified fee market across heterogeneous workloads fails in three concrete ways (high-volume schema absorbs unrelated spikes, low-volume schema cannot trigger its own price discovery, aggregate base fee fits no profile). Per-schema fees fixes each.

The paper’s four contributions resolve the design space the introduction posed. (1) **Mechanism (§4)**: per-schema base-fee adjustment with EIP-1559 dynamics, per-schema tip auction within slot allocation, configurable burn-versus-routing split, integration with PoUA’s adaptive  $\tau_{\text{burn}}$  re-base. (2) **Cost-to-grind preservation theorem (§5.1)**: even at the most adversary-friendly per-schema routing ( $\rho_\sigma = 0.5$  chain-wide), PoUA’s Lemma 1 floor holds with the same constants. The chain’s economic-security floor is unchanged by introducing per-schema fees. (3) **Comparative analysis (§8)**: per-schema fees occupies a distinct design point against EIP-1559, Solana priority fees, Cosmos `x/feemarket`, and Aptos gas pricing, by being the only mechanism in the comparison that treats workload type as the unit of pricing. (4) **Calibration recommendation (§4.2, §7.2)**: governance-tuned defaults per demand profile, with the chain enforcing protocol-level bounds on  $T_\sigma$  and  $\rho_\sigma$ .

The mechanism composes orthogonally with two adjacent primitives. **Sponsored gas** (Iris §7) gets

*better* economics under per-schema fees because per-schema isolation reduces fee variance compared to a unified market. **Native delegation** (companion paper) is unaffected by per-schema fees on the signing side, and the fee-payer field is independent of the signer field by construction. The composition is clean enough that Iris’s commercial viability becomes a function of per-schema fee dynamics rather than a function of chain-wide unified-fee volatility.

**What ships in v0.** Per-schema state per registered schema, EIP-1559-style adjustment per schema, three pre-configured profile defaults (high-volume / high-value / bursty) plus custom-parameter registration. Recommended  $\xi = 1/8$ , default  $T_\sigma = 0.5$ , default  $\rho_\sigma = 0$ . Sovereign SDK integration is three extensions (§7.1); state-tree cost is negligible (~64B per schema); compute cost is  $O(|\Sigma_{\text{block}}|)$  per block.

**What we are watching.** The §9 limitations name five forward-looking extensions: empirical calibration refinement once devnet data lands, generalized sponsorship-model analysis, multi-resource within a schema, cross-chain fee-market portability, and separated-proposer-builder architectures. Each is separable; each can ship as a v0.3 or later increment without breaking v0.2 fee state.

**Invitations.** The paper, the planned simulator (`prototypes/per-schema-fees-sim/` post-paper), and the chain implementation are open to external review. Cold-asks for §5.1 theorem review are open through the PoUA reviewer channel at `hello@ligate.io`. Per-schema fee dynamics are a chain economic-design problem with reach beyond Ligate; feedback from fee-market researchers in the Cosmos, Ethereum, and Aptos ecosystems is especially welcome.

The §1.4 central question was: how does an attestation-native chain price attestation work in a way that respects per-schema demand heterogeneity, while preserving PoUA’s cost-to-grind security floor and composing cleanly with sponsored-gas and native-delegation patterns? This paper answers: per-schema EIP-1559 dynamics with a configurable burn-versus-routing split, coupled to PoUA’s chain-wide  $\tau_{\text{burn}}$ . The mechanism is small, the security argument is tight, and the composition with adjacent primitives is orthogonal. Per-schema fees is the fee-market primitive attestation chains have been waiting for.

---

## 11. Frequently Asked Questions

### Q1. Why per-schema and not multi-resource (gas / storage / DA as separate axes)?

Multi-resource fee markets price the chain’s bottleneck resources separately. They work well when the bottleneck shifts (Ethereum’s blob fees vs execution gas). Per-schema fees prices the chain’s *workload types* separately. They work well when the workloads have orders-of-magnitude different demand profiles. The two are not mutually exclusive: a future extension (§9.3) could price sub-resources within a schema, layered on top of per-schema dynamics. v0 ships per-schema as the primary axis because that is the axis where attestation-native chains differ most from single-workload chains.

### Q2. How does this interact with EIP-1559 if Ligate later bridges to Ethereum?

For bridged transactions (Ligate-to-Ethereum or Ethereum-to-Ligate), each chain prices its own block space with its own mechanism. There is no cross-chain fee-market interaction; the bridge transaction pays Ligate’s per-schema fee on Ligate, and pays Ethereum’s EIP-1559 fee on Ethereum. The cross-chain fee question only arises if a schema is recognized on both chains and the chain

needs to price *attestation work* in a portable way. That is §9.4 (cross-chain fee-market portability), explicitly out of scope for v0.2.

**Q3. What if a schema has near-zero traffic?**

Near-zero traffic schemas sit at  $u_\sigma \rightarrow 0$ , which drives  $b_\sigma$  toward  $b_\sigma^{\min}$  asymptotically. The min-clip prevents the base fee from going below the registrant’s configured floor. The schema’s per-block slot allocation  $w_\sigma$  is still reserved (it sits unused); the slot capacity could be reallocated via governance if a schema is persistently below allocation. v0 ships fixed-allocation; governance can tune.

**Q4. Doesn’t this just reintroduce the unified-fee-market problems at the schema level?**

No, for two reasons. First, within a schema, demand is by construction homogeneous (it is one workload type by definition); EIP-1559 dynamics work in this case. Second, the cross-schema interaction is bounded to the slot allocation and the §A.1 detector; per-schema dynamics do not bleed into each other except through the chain-wide  $\tau_{\text{burn}}$  rebase, which is the intended bridge to economic security.

**Q5. Why is  $\rho_\sigma$  capped at 0.5 and not 1.0?**

Two reasons. First, the §5.1 cost-to-grind preservation theorem requires the validator’s share of post-burn fee to be strictly positive; if  $\rho_\sigma = 1$ , validators have no economic incentive to include attestations from that schema, and the chain’s basic inclusion incentive breaks. Second, governance preference for a balanced split: 50% routing to the schema author lets the author capture meaningful upside without removing validator alignment with the schema’s success.

**Q6. Can  $\rho_\sigma$  be changed after registration?**

v0 ships immutable  $\rho_\sigma$  at registration. A schema that wants to change  $\rho_\sigma$  must either go through governance (slow) or register a new schema (clean but breaks downstream integrations). v0.3 could specify governance-mediated  $\rho_\sigma$  adjustment with rate-limit; v0.2 keeps the parameter immutable for simplicity.

**Q7. What happens to attestations that arrive when  $u_\sigma > T_\sigma$  but the validator has spare block capacity in other schemas’ slots?**

Slot allocation is strict in v0: a schema’s attestations cannot use another schema’s slots. The attestation waits in the mempool for the next block. v1+ may relax this (allow cross-schema slot borrowing with a fee penalty), which would soften the slot caps; v0 keeps strict allocation for analysis tractability.

**Q8. Does this require schemas to predict their own demand profile correctly?**

Schemas declare  $T_\sigma$  and  $\rho_\sigma$  at registration as a *starting point*. The base fee  $b_\sigma$  dynamically adjusts to actual demand regardless of the registrant’s prediction. If a schema author predicts low-volume (sets  $T_\sigma = 0.7$ ) but actual demand is high-volume, the base fee climbs faster than under the default  $T_\sigma = 0.5$ ; users feel it, the schema author can propose a governance adjustment. The starting parameters are not load-bearing for correctness, only for fee-market behavior near launch.

---

## References

EIP-1559 and Ethereum fee markets.

- Buterin, V., Conner, E., Dudley, R., Slipper, M., Norden, I., Bakhta, A. (2019). *EIP-1559: Fee market change for ETH 1.0 chain*. Ethereum Improvement Proposal. <https://eips.ethereum.org/EIPS/eip-1559>
- Roughgarden, T. (2021). *Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559*. <https://arxiv.org/abs/2012.00854>

#### **Multi-resource and per-workload fee markets.**

- Solana Labs (2022+). *Priority fees on Solana*. Solana documentation. <https://solana.com/docs/core/fees>
- Cosmos SDK Authors (2024+). *`x/feemarket` module in Cosmos SDK*. <https://docs.cosmos.network/main/build/modules/feemarket>
- Aptos Foundation (2023+). *Gas and pricing on Aptos*. <https://aptos.dev/concepts/gas-txn-fee>
- Buterin, V., Whitehat, B. et al. (2024). *EIP-4844: Shard Blob Transactions*. <https://eips.ethereum.org/EIPS/eip-4844>

#### **Attestation chains.**

- Ethereum Attestation Service. <https://attest.org/>
- Ceramic Network. <https://ceramic.network/>

#### **Restaking and per-AVS economics.**

- Drake, J., Buterin, V., Edgington, B., Feist, D. et al. (2023). *EigenLayer: The Restaking Collective*. <https://docs.eigenlayer.xyz/eigenlayer/overview/whitepaper>

#### **Companion Ligate Labs research.**

- Ligate Labs (2026). *Proof of Useful Attestation: Consensus-Weighting Primitive for Attestation-Native Chains*. Working paper v0.8.
- Ligate Labs (2026). *Native Delegation as a Runtime Primitive*. Working paper v0.2.
- Ligate Labs (2026). *Schema-Bound Tokens: AttestorSet as Mint Authority*. Working paper v0.1.

#### **Chain stack.**

- Sovereign Labs (2024). *Sovereign SDK*. <https://github.com/Sovereign-Labs/sovereign-sdk>
- Celestia Labs (2023). *Celestia: Modular Data Availability*. <https://celestia.org/learn/>
- Inter-Blockchain Communication (IBC) protocol specification. <https://github.com/cosmos/ibc>

#### **Implementation references.**

- Ligate Chain implementation. <https://github.com/ligate-io/ligate-chain> (per-schema fees milestone tracked in chain repo).
- Planned per-schema-fees simulator. To live at `prototypes/per-schema-fees-sim/` in `ligate-research` once milestone M1 begins.

## Appendix A: Recommended v0 Parameter Defaults

For implementer convenience, the recommended v0 protocol parameter defaults from §4.2 and §7.2 in one place:

| Parameter                            | Default                   | Bound              | Notes                                   |
|--------------------------------------|---------------------------|--------------------|-----------------------------------------|
| $\xi$ (adjustment rate)              | 1/8                       | $[1/16, 1/4]$      | per-block clip,<br>matches EIP-1559     |
| $T_\sigma$ (target utilization)      | 0.5                       | $[0.1, 0.9]$       | declared at<br>registration             |
| $\rho_\sigma$ (routing fraction)     | 0                         | $[0, 0.5]$         | declared at<br>registration             |
| $\tau_\sigma^{\min}$ (tip floor)     | 0                         | $\geq 0$           | declared at<br>registration             |
| $b_\sigma^{\min}$ (base-fee floor)   | 1 uavow                   | governance-tunable | 1 micro-unit                            |
| $b_\sigma^{\max}$ (base-fee ceiling) | $10^{12}$ uavow           | governance-tunable | $\sim 10^6$ AVOW per<br>attestation cap |
| $w_\sigma$ (slot allocation)         | proportional to<br>volume | governance         | epoch-rebalanced                        |
| $T_{\text{block}}$ (block time)      | 12 seconds                | governance         | shared with PoUA                        |

Profile defaults (recommended starting points for schema registration):

| Profile                            | $T_\sigma$ | $\rho_\sigma$ | Best for                                  |
|------------------------------------|------------|---------------|-------------------------------------------|
| high-volume                        | 0.5        | 0.2           | Themisra,<br>content-authenticity         |
| high-value                         | 0.7        | 0.4           | Sovereign identity,<br>regulatory filings |
| bursty                             | 0.3        | 0.1           | NFT mints, Iris<br>campaigns              |
| latency-critical<br>(special case) | 0.3        | 0.0           | Iris agent actions,<br>sub-second flows   |

## Appendix B: Formal Definitions

We collect the formal definitions used throughout the paper in one place.

**Definition (Schema).** A tuple  $\sigma = (\text{schema\_id}, A, V, F)$  where  $\text{schema\_id}$  is the canonical hash,  $A$  is the attester set,  $V$  is the validation rule, and  $F = \text{FeeState}(\sigma)$  is the fee-market state defined below.

**Definition (Fee-market state).**  $\text{FeeState}(\sigma) = (b_\sigma, u_\sigma(t), T_\sigma, \rho_\sigma, \tau_\sigma^{\min}, b_\sigma^{\min}, b_\sigma^{\max}, \xi_\sigma)$  where:

- $b_\sigma$ : current base fee, denominated in `uavow`
- $u_\sigma(t)$ : observed utilization at block  $t$ , fraction of allocated slots filled
- $T_\sigma$ : target utilization, declared at registration
- $\rho_\sigma$ : fee-routing fraction
- $\tau_\sigma^{\min}$ : tip floor
- $b_\sigma^{\min}, b_\sigma^{\max}$ : base-fee clip bounds
- $\xi_\sigma$ : per-block max-change rate

**Definition (Base-fee adjustment).** Per-block update:

$$b_\sigma(t+1) = \text{clip}_{[b_\sigma^{\min}, b_\sigma^{\max}]} \left( b_\sigma(t) \cdot \left( 1 + \xi_\sigma \cdot \frac{u_\sigma(t) - T_\sigma}{T_\sigma} \right) \right)$$

**Definition (Burn split).** For an attestation  $\alpha$  of schema  $\sigma$  paying base fee  $b_\sigma$ :

| Destination          | Fraction                                           |
|----------------------|----------------------------------------------------|
| Burn                 | $\tau_{\text{burn}}$                               |
| Schema registrant    | $(1 - \tau_{\text{burn}}) \cdot \rho_\sigma$       |
| Validator (proposer) | $(1 - \tau_{\text{burn}}) \cdot (1 - \rho_\sigma)$ |

Plus the tip  $\tau_\alpha$  to the validator.

**Definition (Validator income).** For block  $B$  at slot  $t$ :

$$R_v(B, t) = R_b + \sum_{\alpha \in B} \tau_\alpha + \sum_{\alpha \in B} (1 - \tau_{\text{burn}}) \cdot (1 - \rho_{\sigma(\alpha)}) \cdot b_{\sigma(\alpha)} \cdot |\alpha|$$

**Definition (Cost-to-grind).** Per PoUA §5.5.3 Lemma 1, the non-recoverable cost an adversary must pay to inflate their reputation by  $\Delta r$  through legitimate-looking attestation work:

$$F_{\text{net}} \geq \tau_{\text{burn}} \cdot \frac{\Delta r}{\eta \cdot \alpha_{\text{eff}}}$$

where  $\eta$  is the adversary's coalition share of validator power and  $\alpha_{\text{eff}}$  is the effective reputation-per-fee ratio under the coalition's attestation pattern. Per §5.1 of this paper, the bound holds per-schema under per-schema fees with arbitrary  $\rho_\sigma \leq 0.5$ .